

ACVP Dry Run Plan

Module: pqc_crypto_module v0.1.0 **Date:** 2026-04-28

Disclaimer: This package does not imply FIPS 140-3 validation or CMVP certification.

1. Purpose

This document defines the plan for integrating NIST ACVP (Automated Cryptographic Validation Protocol) test vectors into pqc_crypto_module. ACVP validation is mandatory for CMVP certification. This dry run exercises the module against the expected vector formats before engaging the ACVP server.

2. Algorithms in Scope

Algorithm	FIPS Standard	ACVP Spec	Status
ML-DSA-65	FIPS 204	ACVP-ML-DSA (draft)	Functional — 5 signThenVerify vectors pass
ML-KEM-768	FIPS 203	ACVP-ML-KEM (draft)	Functional — 5 encapsThenDecaps vectors pass (F-001 CLOSED)
SHA3-256	FIPS 202	ACVP-SHA3	Functional — 5 known-answer vectors pass

3. ML-DSA-65 Test Vectors

3.1 KeyGen

ACVP request format:

```

{
  "vsId": 1,
  "algorithm": "ML-DSA",
  "mode": "keyGen",
  "revision": "FIPS204",
  "testGroups": [
    {
      "tgId": 1,
      "testType": "AFT",
      "parameterSet": "ML-DSA-65",
      "tests": [
        {
          "tcId": 1,
          "seed": "<hex-encoded 32-byte seed>"
        }
      ]
    }
  ]
}

```

Expected response format:

```

{
  "vsId": 1,
  "testGroups": [
    {
      "tgId": 1,
      "tests": [
        {
          "tcId": 1,
          "pk": "<hex-encoded public key>",
          "sk": "<hex-encoded private key>"
        }
      ]
    }
  ]
}

```

Deterministic mode requirement: The module must accept a fixed seed and produce deterministic key pairs. This requires a test-only code path that bypasses OsRng and injects the ACVP-provided seed.

3.2 SigGen

ACVP request format:

```

{
  "vsId": 2,
  "algorithm": "ML-DSA",
  "mode": "sigGen",
  "revision": "FIPS204",
  "testGroups": [
    {

```

```

    "tgId": 1,
    "testType": "AFT",
    "parameterSet": "ML-DSA-65",
    "deterministic": true,
    "tests": [
      {
        "tcId": 1,
        "message": "<hex-encoded message>",
        "sk": "<hex-encoded private key>",
        "rnd": "<hex-encoded 32-byte randomness or empty for
deterministic>"
      }
    ]
  }
}

```

Expected response format:

```

{
  "vsId": 2,
  "testGroups": [
    {
      "tgId": 1,
      "tests": [
        {
          "tcId": 1,
          "signature": "<hex-encoded signature>"
        }
      ]
    }
  ]
}

```

3.3 SigVer

ACVP request format:

```

{
  "vsId": 3,
  "algorithm": "ML-DSA",
  "mode": "sigVer",
  "revision": "FIPS204",
  "testGroups": [
    {
      "tgId": 1,
      "testType": "AFT",
      "parameterSet": "ML-DSA-65",
      "tests": [
        {
          "tcId": 1,
          "message": "<hex-encoded message>",
          "pk": "<hex-encoded public key>",

```

```

        "signature": "<hex-encoded signature>"
    }
  ]
}

```

Expected response format:

```

{
  "vsId": 3,
  "testGroups": [
    {
      "tgId": 1,
      "tests": [
        {
          "tcId": 1,
          "testPassed": true
        }
      ]
    }
  ]
}

```

4. ML-KEM-768 Test Vectors

Resolved: ML-KEM-768 is now fully functional via pqcrypto-mlkem (F-001 CLOSED).

4.1 KeyGen

```

{
  "vsId": 4,
  "algorithm": "ML-KEM",
  "mode": "keyGen",
  "revision": "FIPS203",
  "testGroups": [
    {
      "tgId": 1,
      "testType": "AFT",
      "parameterSet": "ML-KEM-768",
      "tests": [
        {
          "tcId": 1,
          "z": "<hex-encoded 32-byte seed>",
          "d": "<hex-encoded 32-byte seed>"
        }
      ]
    }
  ]
}

```

```
]
}
```

4.2 Encapsulation

```
{
  "vsId": 5,
  "algorithm": "ML-KEM",
  "mode": "encapDecap",
  "revision": "FIPS203",
  "testGroups": [
    {
      "tgId": 1,
      "testType": "AFT",
      "function": "encapsulation",
      "parameterSet": "ML-KEM-768",
      "tests": [
        {
          "tcId": 1,
          "ek": "<hex-encoded encapsulation key>",
          "m": "<hex-encoded 32-byte randomness>"
        }
      ]
    }
  ]
}
```

4.3 Decapsulation

```
{
  "vsId": 6,
  "algorithm": "ML-KEM",
  "mode": "encapDecap",
  "revision": "FIPS203",
  "testGroups": [
    {
      "tgId": 1,
      "testType": "VAL",
      "function": "decapsulation",
      "parameterSet": "ML-KEM-768",
      "tests": [
        {
          "tcId": 1,
          "dk": "<hex-encoded decapsulation key>",
          "c": "<hex-encoded ciphertext>"
        }
      ]
    }
  ]
}
```

5. SHA3-256 Test Vectors

5.1 Hash (AFT — short/long messages)

```
{
  "vsId": 7,
  "algorithm": "SHA3-256",
  "revision": "1.0",
  "testGroups": [
    {
      "tgId": 1,
      "testType": "AFT",
      "tests": [
        {
          "tcId": 1,
          "msg": "<hex-encoded message>",
          "len": 256
        }
      ]
    }
  ]
}
```

Expected response:

```
{
  "vsId": 7,
  "testGroups": [
    {
      "tgId": 1,
      "tests": [
        {
          "tcId": 1,
          "md": "<hex-encoded 32-byte digest>"
        }
      ]
    }
  ]
}
```

5.2 Monte Carlo Test (MCT)

```
{
  "testGroups": [
    {
      "tgId": 2,
      "testType": "MCT",
      "tests": [
        {
          "tcId": 100,
          "msg": "<hex-encoded initial seed>",

```

```

    "len": 256
  }
]
}
]
}

```

MCT requires 100 iterations of the inner loop (1000 hashes per iteration) returning intermediate results.

6. Deterministic Test Mode Requirements

To replay ACVP vectors, the module must support a deterministic mode:

1. **Seed injection:** A `#[cfg(test)]` or feature-gated code path that accepts a fixed seed instead of calling `OsRng`.
 2. **No side effects:** Deterministic mode must not affect the FSM state or self-test results.
 3. **Isolation:** The deterministic RNG must only be usable in test builds. Production builds must have no path to inject a seed.
 4. **Implementation approach:**
 - Add a `DeterministicRng` struct implementing `RngCore + CryptoRng` behind `#[cfg(test)]`.
 - Modify `keygen` and `sign` functions to accept a generic `R: CryptoRng + RngCore` parameter.
 - Test harness injects `DeterministicRng`; production code injects `OsRng`.
-

7. Known Gaps

Gap	Impact	Resolution path
Official ACVP vectors not yet available for ML-DSA/ML-KEM in all modes	Cannot complete official validation	Monitor NIST ACVP server; use draft vectors from reference implementations
No tools/ acvp_dry_run/ directory	RESOLVED	Harness built with SHA3, ML-DSA, ML-KEM runners and JSON vector parsing
ML-KEM is placeholder	RESOLVED (F-001 CLOSED)	pqcrypto-mlkem integrated with roundtrip verification
Deterministic mode not implemented	Cannot replay keygen/siggen vectors	Implement generic RNG parameter pattern (future work for official ACVP)

8. Tooling Plan

8.1 tools/acvp_dry_run/

Create a standalone Rust binary crate:

```
tools/acvp_dry_run/  
Cargo.toml  
src/  
  main.rs          # CLI entry point  
  parser.rs        # ACVP JSON parser  
  runner.rs        # Calls pqcrypto_module functions  
  comparator.rs    # Compares module output to expected values  
vectors/  
  ml_dsa_65/       # JSON vector files  
  ml_kem_768/      # JSON vector files  
  sha3_256/        # JSON vector files
```

CLI interface:

Run all vectors

```
cargo run --bin acvp-dry-run -- --vectors-dir vectors/
```

Run specific algorithm

```
cargo run --bin acvp-dry-run -- --algorithm ML-DSA-65 --vectors-  
dir vectors/ml_dsa_65/
```

Output results as JSON

```
cargo run --bin acvp-dry-run -- --output results.json
```

8.2 Vector Sources

Source	URL	Notes
NIST ACVP Server	https://demo.acvts.nist.gov/	Official; requires registration
pqc-certificates project	https://github.com/IETF-Hackathon/pqc-certificates	Community vectors
Reference implementation KATs	Bundled with pqcrypto source	Useful for initial dry run

9. Success Criteria



tools/acvp_dry_run/ parses all three algorithm vector formats



SHA3-256 known-answer vectors pass (5/5)



- ☒ ML-DSA-65 sign-then-verify vectors pass (5/5)
 - ☒ ML-KEM-768 encaps-then-decaps vectors pass (5/5, F-001 CLOSED)
 - ☒ All results exportable as JSON (report.json)
 - ☐ Zero discrepancies between module output and reference vectors (15/15)
 - ☐ Deterministic mode for keygen/siggen replay (future: official ACVP submission)
-

End of ACVP dry run plan.